ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТ 1
ТАЙЛАН

Үйлдлийн систем /F.CSM302/

Сэдэв: ”Designing a Thread Pool. Programming Project 1 (7-р бүлэг), Worker activity”

Хуваарилалтын код: IV-2-9

Хичээл заасан багш: Г.Ганбат /F.CS10/ .....

Гүйцэтгэсэн баг: Ц.Номинжин /B241940137/ .....
Е.Айсауле /B221940407/ .....
Б.Балжинням /B231940033/ .....
М.Бат-Орших /B200930061/ .....
Г.Хулан /B241940027/ .....
Э.Энхсайхан /B242270097/ .....

Улаанбаатар хот

2026 он

ХУРААНГУЙ

Асуудал. Хүсэлт бүрт шинэ thread үүсгэх нь үүсгэх, устгах зардалтай бөгөөд нэгэн зэрэг ажиллах thread-ийн тоог хязгаарладаггүй. Thread pool нь цөөн worker thread-ийг урьдчилан үүсгэж, ажлыг дундын queue-ээр хуваарилна. Энэ queue-д client болон worker-ууд зэрэг хандах тул race condition үүсэх, ажилгүй worker CPU дэмий үрэх аюултай.

Шийдэл. Сурах бичгийн Programming Project 1-ийн POSIX хувилбарын дагуу C хэл, POSIX thread (pthreads), mutex lock болон counting semaphore ашиглан `pool_init`, `pool_submit`, `pool_shutdown` API-тай thread pool хэрэгжүүлэв. Программ үйлдэл бүрээ микросекундын нарийвчлалтай JSON trace болгон гаргана. Браузерийн интерфейс уг программыг бодитоор ажиллуулж, багшийн оноосон ”Worker activity” дүрслэлээр worker бүрийн idle, running, completed төлөв, queue-ийн дүүргэлт, татгалзлыг алхам алхмаар харуулна.

Туршилт. Программыг 29 автомат тестээр (номын жишээ, үндсэн, захын, буруу оролт) шалгав. Тест бүрийн trace-ийг C кодоос хараат бус Python шалгагчаар дахин шалгаж, ThreadSanitizer-ээр race condition хайв. Worker-ийн тоо, queue-ийн багтаамж, дүүрэх үеийн бодлогын нөлөөг гурван туршилтаар хэмжив.

Гол үр дүн. 29/29 тест амжилттай. Бүх ажиллуулалтад task бүр яг нэг удаа гүйцэтгэгдэж, queue FIFO дарааллаар ажиллаж, багтаамжаас хэтрээгүй. ThreadSanitizer race илрүүлээгүй. 40 task-ийн ачаалалд worker-ийн тоог 1-ээс 16 болгоход нийт хугацаа 8377 мс-ээс 647 мс болж 13.0 дахин хурдассан.

ГАРЧИГ

Хураангуй 1

НЭГ. *Удиртгал* 4

1.1 Зорилго 4

1.2 Ач холбогдол 4

1.3 Хамрах хүрээ 4

ХОЁР. *Сурах бичгийн шаардлага* 4

2.1 Programming Project 1-ийн үндсэн шаардлага 4

2.2 Багшийн нэмэлт шаардлага 5

2.3 Шаардлагын хангалт 5

ГУРАВ. *Онолын үндэс* 6

3.1 Thread pool 6

3.2 Critical section ба race condition 6

3.3 Mutex lock 6

3.4 Semaphore 6

3.5 Bounded buffer ба тойрог queue 7

3.6 Thread-ийг зогсоох 7

3.7 Invariant 7

ДӨРӨВ. *Системийн зохиомж* 7

4.1 Бүрэлдэхүүн хэсгүүд 7

4.2 Өгөгдлийн урсгал 8

4.3 Үйлдлийн (event) формат 8

4.4 Хязгаар ба хамгаалалт 9

ТАВ. *Хэрэгжилт* 10

5.1 Хэл, сан, орчин 10

5.2 C core: queue 10

5.3 C core: worker 10

5.4 C core: submit ба shutdown 12

5.5 Сервер 13

5.6 Интерфейс 13
5.7 GitHub дээрх ажиллагаа 14
5.8 Source 14

ЗУРГАА. *Туршилт ба UI дүрслэл* 14
6.1 Өөрчилж турших параметр 14
6.2 Юуг ажиглах 15
6.3 Бэлэн туршилтууд 16

ДОЛОО. *Туршилтын арга* 16
7.1 Орчин 16
7.2 Core-ийн автомат тест 16
7.3 Trace шалгагч ба ThreadSanitizer 17
7.4 Гүйцэтгэлийн туршилт 17
7.5 Гар туршилт 17

НАЙМ. *Үр дүн* 17
8.1 Core-ийн тестийн үр дүн 17
8.2 Гүйцэтгэлийн туршилтын үр дүн 18
8.3 Интерфейсийн үр дүн 19
8.4 Хэрэглэгчийн компьютер дээрх баталгаажуулалт 24

ЕС. *Хэлэлцүүлэг* 29
9.1 Үр дүнгийн тайлбар 29
9.2 Хүлээх дараалал ба deadlock 29
9.3 Хязгаарлалт 29

АРАВ. *Дүгнэлт* 29

Ашигласан материал 30

Хавсралт А. Ажлын төлөвлөгөө 31

НЭГ. УДИРТГАЛ

1.1 Зорилго

Сурах бичгийн 7-р бүлгийн Programming Project 1 ”Designing a Thread Pool”-ийг хэрэгжүүлж, хэрэглэгч worker-ийн тоо, queue-ийн багтаамж, ажлын параметрийг өөрчилж турших боломжтой программ бүтээх. Программын ажиллагааг багшийн оноосон ”Worker activity” дүрслэлээр харуулж, хэрэгжилт болон туршилтын үр дүнг баримтжуулах.

1.2 Ач холбогдол

Thread pool нь вэб сервер, өгөгдлийн сангийн сервер, Java-ийн `ExecutorService`, Linux kernel-ийн `workqueue` зэрэг олон системийн үндэс юм. Энэ төсөл нь mutex, semaphore, bounded buffer (producer-consumer) ойлголтыг нэг бодит программд нэгтгэнэ: task өгөгч client нь producer, worker-ууд нь consumer болно. Thread-үүдийн ажиллагаа нүдэнд харагддаггүй тул worker бүрийн төлөвийг хугацааны тэнхлэг дээр дүрслэх нь ойлголтыг ихээхэн хөнгөвчилнө.

1.3 Хамрах хүрээ

- C хэл дээрх thread pool: тойрог массив queue, `pthread_mutex`, `sem_t` semaphore, worker thread (сурах бичгийн үндсэн шаардлага).
- Номын API-г өөрчлөлтгүй хадгалж, worker-ийн тоо болон queue-ийн багтаамжийг тохируулах өргөтгөл.
- Оролтыг шалгах, үйлдэл бүрийг JSON trace болгон хэвлэх, ажиллаж дууссаны дараа 8 инвариант шалгах.
- Бодит программтай холбогдсон браузерийн интерфейс: тохиргоо, Worker activity дүрслэл, timeline, үйл явдлын бүртгэл.
- Автомат тест, ThreadSanitizer, CI, хэрэглэгчийн ба operational гарын авлага (Manual).

Java хувилбар (номонд сонголтоор өгсөн), task-ийн priority, worker-ийн тоог ажиллах явцад динамикаар өөрчлөх хамрах хүрээнд ороогүй.

ХОЁР. СУРАХ БИЧГИЙН ШААРДЛАГА

2.1 Programming Project 1-ийн үндсэн шаардлага

Сурах бичгийн (Silberschatz, Galvin, Gagne, *Operating System Concepts*, 10-р хэвлэл) 7-р бүлгийн Project 1 нь client-ийн өгсөн ажлыг thread pool-ийн worker-уудаар гүйцэтгэхийг шаардана. POSIX хувилбарын шаардлага:

1. `pool_init()` нь worker thread-үүдийг эхэнд нь үүсгэж, mutex болон semaphore-ийг эхлүүлнэ.

2. `pool_submit()` нь функц болон өгөгдлийг `task` бүтэц болгож `enqueue()`-ээр queue-д хийнэ. Queue массив бөгөөд дүүрсэн бол 1, амжилттай бол 0 буцаана.
3. `worker()` нь ажил ирэхийг semaphore дээр хүлээж, `dequeue()`-ээр авсан task-ийг `execute()`-ээр гүйцэтгэнэ.
4. Queue-д хандах бүх үйлдлийг mutex lock-оор хамгаална.
5. `pool_shutdown()` нь worker-уудыг зогсоож `pthread_join()` хийнэ. Номын `client.c` жишээ ажиллах ёстой.

2.2 Багшийн нэмэлт шаардлага

Бие даалт 1-ийн удирдамжаар дараах шаардлага нэмэгдэнэ: оролт, параметрийг өөрчилж дахин турших; буруу болон захын оролтыг шалгаж ойлгомжтой мэдээлэх; тохиргоог солих, дахин ажиллуулах, reset хийх; оноосон ”Worker activity” UI-г бодит ажиллагаатай холбох; номын болон нэмэлт туршилтаар зөв ажиллагааг нотлох; Manual-ын дагуу өөр компьютерт ажиллуулах боломжтой байх; GitHub repository, README, Release/Tag. Хуваарилалтын файлд 9-р сэдвийн хүрэх үр дүнг ”Worker-ийн тоо, task queue, ажлын параметрийг өөрчилж, worker бүрийн idle, running, completed төлөвийг хугацааны явцад харуулна” гэж тодорхойлсон.

2.3 Шаардлагын хангалт

Хүснэгт 1. Шаардлага ба хэрэгжсэн газар

Шаардлага	Хэрэгжилт
`pool_init`: thread, mutex, semaphore	`src/threadpool.c`: `pool_init()` (3 worker, queue 10), `pool_init_ex(n, cap)`
`pool_submit`, массив queue, дүүрвэл 1	Тойрог массив, `POOL_OK=0`, `POOL_FULL=1`
`worker`: semaphore дээр хүлээх	`sem_wait(work_available)` → `dequeue()` → `execute()`
Queue-г mutex-ээр хамгаалах	`pthread_mutex_t qmutex`
`pool_shutdown`, `pthread_join`	Shutdown флаг + worker бүрт нэг sem_post, үлдсэн task-уудыг гүйцээгээд гарна
Номын `client.c`	`examples/client.c`, тест T1
Параметр өөрчилж дахин турших, reset	Интерфейсийн тохиргоо, Ажиллуулах/Reset/Анхдагч, бэлэн тест; CLI сонголтууд
Буруу оролт шалгах	C программ (exit 2 + тайлбар), сервер (HTTP 400), UI (улаан талбар)
Worker activity, бодит ажиллагаатай холбох	Бодит C программын trace-ээр worker карт, queue, timeline зурна

Шаардлага	Хэрэгжилт
Нэмэлт туршилт	29 автомат тест, хараат бус trace шалгагч, ThreadSanitizer, 3 туршилт, CI

ГУРАВ. ОНОЛЫН ҮНДЭС

3.1 Thread pool

Thread pool нь эхлэхдээ тогтмол тооны thread үүсгэж, тэднийг pool-д хүлээлгэнэ. Ирсэн ажил queue-д орж, чөлөөтэй thread түүнийг авч гүйцэтгээд дахин pool руу буцна (номын 4.5.1). Давуу тал нь: байгаа thread-ээр үйлчлэх нь шинэ thread үүсгэхээс хурдан; нэгэн зэрэг ажиллах thread-ийн тоо хязгаарлагдмал тул систем хэт ачаалагдахгүй; ажлыг гүйцэтгэх механизмаас тусгаарласан тул хуваарилах стратегийг тусад нь өөрчилж болно.

3.2 Critical section ба race condition

Хэд хэдэн thread нэг дундын өгөгдлийг зэрэг өөрчлөх үед үр дүн нь гүйцэтгэлийн дараалалаас хамаарах нөхцөлийг race condition гэнэ. Queue-д task нэмэх нь ”`queue[tail] = t`”, ”`tail = (tail + 1) % cap`”, ”`count++`” гэсэн хэд хэдэн алхмаас бүрдэх critical section юм. Хоёр worker зэрэг dequeue хийвэл нэг task-ийг давхар авч, нөгөөг алгасах аюултай. Critical section-ийн шийдэл нь харилцан гадуурхал (mutual exclusion), ахиц (progress), хязгаартай хүлээлт (bounded waiting) гэсэн гурван шаардлагыг хангах ёстой.

3.3 Mutex lock

Mutex нь нэг мөчид зөвхөн нэг thread-д critical section-д орох эрх олгоно (`pthread_mutex_lock/unlock`). Цоож барьсан thread л queue-г өөрчлөх тул `head`, `tail`, `count` зөрөхгүй. Цоожийг аль болох богино хугацаанд барих ёстой: task-ийг гүйцэтгэх хугацаанд барьвал worker-ууд дараалж ажиллан pool утгагүй болно.

3.4 Semaphore

Semaphore нь бүхэл тоон утгатай хувьсагч бөгөөд зөвхөн хоёр атомар үйлдлээр хандана. `wait()` (POSIX: `sem_wait`) утгыг 1-ээр хасна; утга 0 байвал thread блоклогдож хүлээнэ. `signal()` (`sem_post`) утгыг 1-ээр нэмж, хүлээж буй thread байвал нэгийг сэрээнэ. Thread pool-д counting semaphore-ийн утга нь queue дахь task-ийн тоотой тэнцүү: ажилгүй worker `sem_wait` дээр унтаж, CPU үрэхгүй (busy waiting байхгүй). Номд unnamed semaphore (`sem_init(&sem, 0, 0)`) ашиглахыг зөвлөсөн.

3.5 Bounded buffer ба тойрог queue

Thread pool-ийн queue нь номын 7.1.1 хэсгийн bounded buffer бодлогын нэг хэлбэр. Номын шийдэл `mutex`, `empty`, `full` гурван синхрончлолын хувьсагч ашигладаг (номын Зураг 7.1–7.2):

Код 1. Consumer-ийн бүтэц (номын Зураг 7.2)

```
while (true) {
   wait(full);
   wait(mutex);
   /* remove an item from buffer to next_consumed */
   signal(mutex);
   signal(empty);
   /* consume the item in next_consumed */
}
```

Thread pool-д worker нь consumer бөгөөд `full`-ийн үүргийг `work_available` semaphore гүйцэтгэнэ. Харин `empty`-ийн оронд дүүрсэн queue-д client-ийг блоклохгүйгээр шууд 1 буцаадаг нь номын `pool_submit()`-ийн тодорхойлолт юм. Queue нь `cap` нүдтэй массив, хоёр заагчтай: `tail` нь дараагийн тавих нүд, `head` нь дараагийн авах нүд. Заагч бүр `(i + 1) % cap`-ээр урагшилж, сүүлийн нүдний дараа 0 руу эргэнэ. Иймээс анх орсон task анх гүйцэтгэгдэнэ (FIFO).

3.6 Thread-ийг зогсоох

Номд `pthread_cancel()`-ийг санал болгосон (`sem_wait()` нь cancellation point). Гэвч deferred cancellation нь task ажиллаж байх үед `nanosleep` зэрэг cancellation point дээр thread-ийг тасалж, task-ийг дутуу орхих, барьсан нөөцийг суллахгүй байх эрсдэлтэй (номын 4.6.3). Тиймээс хамтран зогсох (cooperative) аргыг сонгов: shutdown флаг тавиад worker бүрт нэг `sem_post` илгээнэ. Энэ нь API-ийн ”бүх task дууссаны дараа зогсоно” гэсэн тайлбартай нийцнэ.

3.7 Invariant

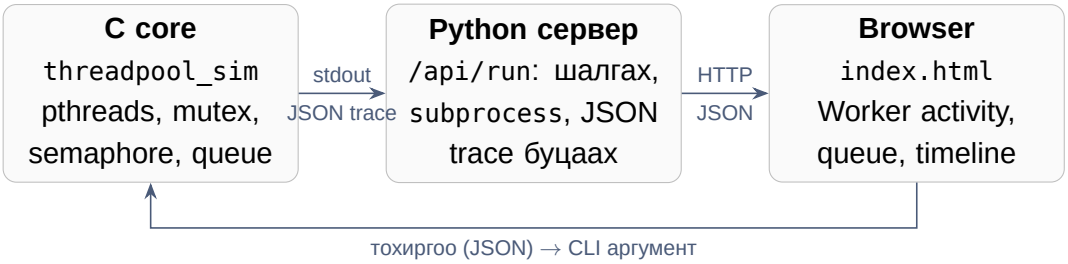
Зөв синхрончлолтой ажиллалтын төгсгөлд ”хүлээн авсан = гүйцэтгэсэн”, ”task бүр яг 1 удаа ажилласан”, ”нэгэн зэрэг ажилласан task ≤ worker-ийн тоо”, ”queue-ийн урт ≤ багтаамж” гэсэн тэнцэл үргэлж биелнэ. Программ төгсгөлд нь эдгээрийг шалгаж (`VERIFY: OK`), тест болон интерфейс зөв ажиллагааны нотолгоо болгон ашиглана.

ДӨРӨВ. СИСТЕМИЙН ЗОХИОМЖ

4.1 Бүрэлдэхүүн хэсгүүд

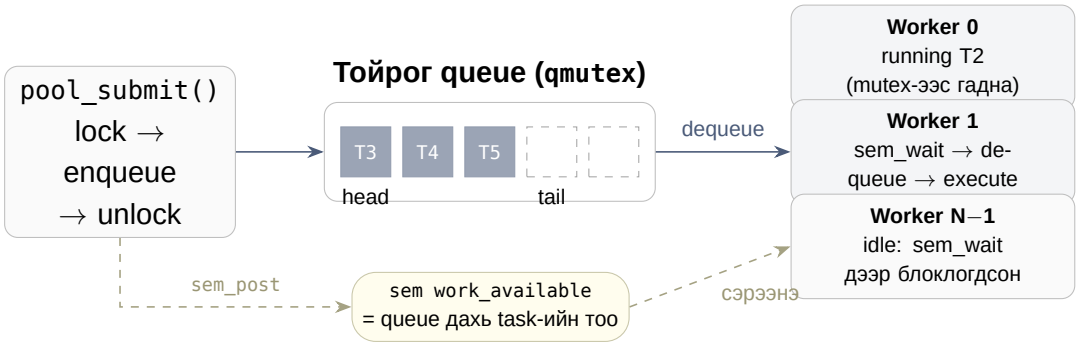
Систем гурван хэсгээс бүрдэнэ (Зураг 1): синхрончлолыг бодитоор гүйцэтгэдэг C программ (core), түүнийг ажиллуулж trace-ийг дамжуулдаг Python сервер, trace-ийг

дүрсэлдэг браузерийн интерфейс.



Зураг 1. Системийн бүрэлдэхүүн ба өгөгдлийн урсгал

Core-ийн дотор client thread task-уудыг queue-д хийж, worker-ууд semaphore-оор сэрж авна (Зураг 2).



Зураг 2. Pool доторх урсгал: submit → queue → semaphore → worker

4.2 Өгөгдлийн урсгал

1. Хэрэглэгч тохиргоо хэсэгт worker, queue, бодлого, task-ийн тоо, хугацаа, ирэх зай, seed-ийг сонгох эсвэл task жагсаалт оруулаад **Ажиллуулах** дарна.
2. Browser `POST /api/run` хүсэлт илгээнэ. Сервер утгуудыг CLI аргумент болгож `build/threadpool_sim`-ийг ажиллуулна. Буруу утгыг C программ өөрөө шалгаж, exit 2 + тайлбар буцаавал сервер HTTP 400 болгон дамжуулна.
3. Core pool-ийг эхлүүлж, task бүрийг тогтоосон агшинд submit хийн, worker-ууд бодитоор гүйцэтгэнэ. Үйлдэл бүр санах ойн trace-д бүртгэгдэнэ. Shutdown-ы дараа 8 инвариант шалгаад бүх trace-ийг JSON болгон stdout руу хэвлэнэ.
4. Browser JSON-ийг хүлээн авч worker карт, queue, timeline, бүртгэлийг зурж, playback-аар тоглуулна. Trace-ийг файлд хадгалж, дараа нь сервергүйгээр дахин үзэж болно.

4.3 Үйлдлийн (event) формат

Trace-ийн үйлдэл бүр нэг мөр бөгөөд тухайн агшны queue-ийн уртыг агуулна (Хүснэгт 2).

Хүснэгт 2. JSON үйлдлийн талбарууд

Талбар	Утга
`t`	`pool_init`-ээс хойшх хугацаа (микросекунд)
`type`	`POOL_INIT`, `IDLE`, `SUBMIT`, `REJECT`, `START`, `END`, `SHUTDOWN`, `WORKER_EXIT`
`w`	worker-ийн дугаар (0-ээс), хамааралгүй бол −1
`task`	task-ийн дугаар, хамааралгүй бол −1
`q`	үйлдлийн дараах queue-ийн урт
`summary`	`submitted`, `rejected`, `completed`, `avg_wait_ms`, `makespan_ms`, `utilization_pct`, `per_worker`
`verify`	`ok` ба 8 шалгалтын нэр, үр дүн, тайлбар

Жишээ мөр:

```
{"t":94,"type":"SUBMIT","w":-1,"task":0,"q":1},
{"t":122,"type":"START","w":1,"task":0,"q":0}
```

Цагийг trace-ийн mutex дотор авдаг тул массивын дараалал хугацааны дараалалтай давхцана. `SUBMIT`, `START`-ийг queue-ийн mutex дор бүртгэдэг тул бүртгэсэн `q` нь бодит queue-ийн урттай яг тэнцүү.

4.4 Хязгаар ба хамгаалалт

С программ: worker 1-64, queue 1-4096, task 1-10000, хугацаа 1-60000 мс, ирэх зай 0-60000 мс. Сервер нэг ажиллуулалтад интерфейсээс 2000-аас олон task, ойролцоогоор 60 секундээс урт ажлыг хүлээж авахгүй, 120 секундэд дуусаагүй процессыг зогсооно, зөвхөн 127.0.0.1 хаягаас хүсэлт хүлээн авна.

ТАВ. ХЭРЭГЖИЛТ

5.1 Хэл, сан, орчин

Хүснэгт 3. Ашигласан технологи

Хэсэг	Технологи
Core	C11 (gcc/clang, `-O2 -Wall -Wextra -pthread`), POSIX threads, POSIX semaphore (macOS дээр `dispatch_semaphore`)
Сервер	Python 3 стандарт сан (`http.server`, `subprocess`), нэмэлт сангүй
Интерфейс	HTML, CSS, JavaScript, SVG; гадны сан, CDN ашиглаагүй
Тест	POSIX shell (`tests/run_tests.sh`), Python 3 (`tests/verify_trace.py`), ThreadSanitizer
CI	GitHub Actions (ubuntu-latest, macos-latest)

5.2 C core: queue

Номын API-г хадгалж, queue-г тойрог массиваар хэрэгжүүлэв. Функцууд `qmutex` барьсан үед л дуудагдана.

Код 2. `enqueue` ба `dequeue` (src/threadpool.c)

```
static int enqueue(task t)
{
    if (qcount == qcap) return 1;
    queue[qtail] = t;
    qtail = (qtail + 1) % qcap;
    qcount++;
    return 0;
}

static task dequeue(void)
{
    task t = queue[qhead];
    qhead = (qhead + 1) % qcap;
    qcount--;
    return t;
}
```

5.3 C core: worker

Worker `sem_wait`-ээр ажил хүлээж, mutex дор dequeue хийгээд, task-ийг mutex-ийн **гадна** гүйцэтгэнэ (Код 3).

Код 3. Worker thread-ийн функц (src/threadpool.c)

```
static void *worker(void *param)
{
    worker_info *me = param;

    for (;;) {
        trace(EV_WORKER_IDLE, me->id, -1, -1);
        psem_wait(&work_available);  /* ажил (эсвэл shutdown) хүлээнэ */

        pthread_mutex_lock(&qmutex);
        if (qcount == 0 && shutting_down) {  /* queue хоосон + shutdown → гарна */
            pthread_mutex_unlock(&qmutex);
            break;
        }
        task t = dequeue();
        trace(EV_START, me->id, t.id, qcount);
        pthread_mutex_unlock(&qmutex);

        execute(t.function, t.data);  /* critical section-оос ГАДНА */

        pthread_mutex_lock(&qmutex);
        completed++;
        per_worker_completed[me->id]++;
        trace(EV_END, me->id, t.id, qcount);
        pthread_mutex_unlock(&qmutex);
    }

    trace(EV_WORKER_EXIT, me->id, -1, -1);
    return NULL;
}
```

Гол шийдлүүд:

- `sem_wait` нь mutex-ээс гадна байна. Mutex барьсан чигээр блоклогдвол client submit хийж чадахгүй тул deadlock болно.
- `execute()`-ийг mutex-ийг суллаад дуудна. Ингэснээр N worker зэрэг ажиллана.
- Queue хоосон, shutdown флаг тавигдсан үед л worker гарна. Иймээс shutdown-ы үед queue-д үлдсэн task-ууд бүгд гүйцэтгэгдэнэ.
- `sem_wait`-ийг `EINTR` (дохиогоор тасалдсан) үед давтана.

5.4 C core: submit ба shutdown

Код 4. `pool_submit` (src/threadpool.c)

```
int pool_submit_id(void (*somefunction)(void *p), void *p, int task_id)
{
    if (!somefunction) return POOL_ERR;

    pthread_mutex_lock(&qmutex);
    if (!initialized || shutting_down) { pthread_mutex_unlock(&qmutex); return
↪   POOL_ERR; }
    if (task_id < 0) task_id = next_auto_id++;

    task t = { somefunction, p, task_id };
    if (enqueue(t) != 0) {
        rejected++;
        trace(EV_REJECT, -1, task_id, qcount);
        pthread_mutex_unlock(&qmutex);
        return POOL_FULL;
    }
    submitted++;
    trace(EV_SUBMIT, -1, task_id, qcount);
    pthread_mutex_unlock(&qmutex);

    psem_post(&work_available);  /* хүлээж буй нэг worker-ийг сэрээнэ */
    return POOL_OK;
}
```

`sem_post`-ийг mutex суллагдсаны дараа дуудна; task queue-д орсон үед л post хийдэг тул semaphore-ийн утга queue-ийн уртаас хэзээ ч их болохгүй.

Код 5. `pool_shutdown`-ийн гол хэсэг (src/threadpool.c, товчилсон)

```
pthread_mutex_lock(&qmutex);
shutting_down = 1;
pthread_mutex_unlock(&qmutex);

/* Worker бүрт нэг "гарах" дохио. Queue-д үлдсэн task бүр өөрийн post-той
 * тул worker-ууд эхлээд бүх task-ийг гүйцээж, дараа нь л гарна. */
for (int i = 0; i < nthreads; i++) psem_post(&work_available);
for (int i = 0; i < nthreads; i++) pthread_join(workers[i].tid, NULL);
```

Зөв байдлын тайлбар. Shutdown-ы өмнө queue-д k task байлаа гэж үзье. Semaphore-ийн утга $k + N$ болно. Worker зөвхөн queue хоосон үед гардаг тул эхний k сэрэлтээр task авч, үлдсэн N сэрэлтээр worker бүр нэг удаа гарна. Иймээс бүх task гүйцэтгэгдэж, `pthread_join` блоклогдож үлдэхгүй. Бусад шийдэл:

- macOS нэргүй semaphore (`sem_init`) дэмждэггүй тул `#ifdef __APPLE__` үед `dispatch_semaphore` ашиглана.
- Trace бүртгэлийг тусдаа `tr_mutex`-ээр хамгаалж, цагийг mutex дотор авдаг тул

үйлдлүүд хугацааны дарааллаар хадгалагдана.

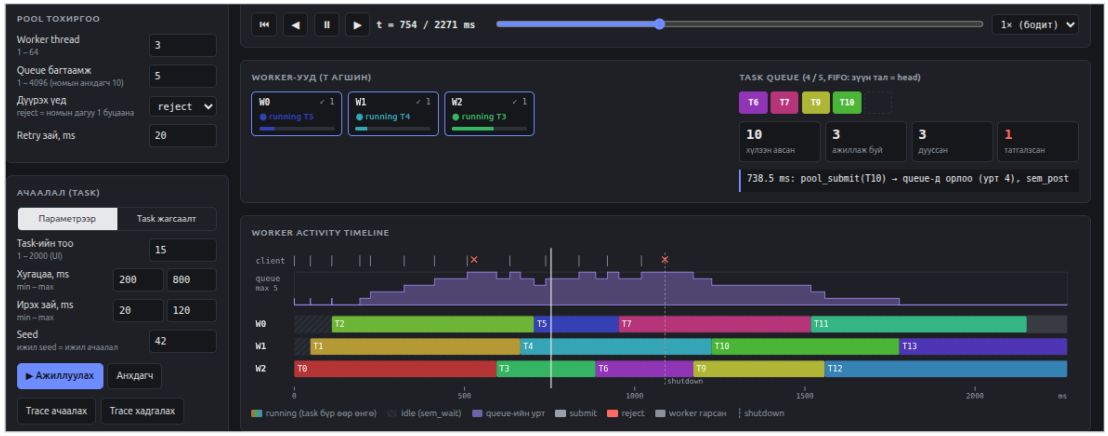
- Аргумент бүрийг `strtol`-оор бүтнээр нь шалгаж (жишээ нь `12abc` хүлээж авахгүй), алдаатай бол Монгол хэлээр тайлбар хэвлээд 2 кодоор гарна. Task файлын алдааг `файл:мөр` байршилтай мэдээлнэ.
- Ажиллаж дууссаны дараа trace-ээс 8 инвариант шалгаж, аль нэг нь зөрвөл 4 кодоор гарна.

5.5 Сервер

`tools/server.py` нь JSON тохиргоог CLI аргумент болгон `threadpool_sim`-ийг `subprocess`-ээр ажиллуулж, stdout дахь JSON trace-ийг browser руу буцаана. Task жагсаалтыг түр файлд бичиж `-f` сонголтоор дамжуулна. C программ 2 кодоор гарвал алдааны текстийг HTTP 400-аар, хугацаа хэтэрвэл HTTP 504-өөр мэдээлнэ. Сервер интерфейсийн файлыг (`ui/index.html`) мөн өөрөө түгээнэ.

5.6 Интерфейс

- **Тохиргоо**: worker, queue, дүүрэх үеийн бодлого (reject/retry), task-ийн тоо, хугацаа, ирэх зай, seed эсвэл task жагсаалт ба бэлэн тест (Зураг 4).
- **Worker карт**: worker бүрийн одоогийн төлөв (● running Tk, ○ idle, ■ exited), явцын шугам, дуусгасан task-ийн тоо (✓ n).
- **Task queue**: хүлээж буй task-ууд FIFO дарааллаар, багтаамжийн хоосон нүд; хүлээн авсан, ажиллаж буй, дууссан, татгалзсан тоолуур; тухайн агшны үйлдлийн тайлбар.
- **Worker activity timeline**: worker бүр нэг мөр, client мөр (submit, reject), queue-ийн уртын график, shutdown, playhead (Зураг 3).
- **Playback**: тоглуулах/зогсоох, өмнөх/дараагийн үйлдэл, гулсуур, 0.25×-10× хурд; timeline дээр дарж, чирж очих.
- **Үр дүн ба шалгалт**, **үйлдлийн бүртгэл**: хэмжүүрүүд, 8 шалгалт, C программын команд ба stderr; бүртгэлийн мөр дээр дарж тэр агшин руу очно.



Зураг 3. Интерфейсийн ерөнхий харагдац: 3 worker, queue 5, 15 task-ийн бодит ажиллалт, t = 754 мс агшин (3 worker running, queue-д 4 task, 1 татгалзал)

5.7 GitHub дээрх ажиллагаа

Repository: `https://github.com/NominjinNomio/F.CSM302-BD_IV-2-9-threadpool`. README (Quick Start), `src/`, `tests/`, `examples/`, `tools/`, `ui/`, `docs/` бүтэцтэй. `.github/workflows/ci.yml` нь push болон Pull Request бүрт Ubuntu дээр build, `make test`, ThreadSanitizer, macOS дээр clang build ба тестийг ажиллуулна. Гишүүн бүрийн хариуцсан хэсэг, commit-ийг Хавсралт А-д тэмдэглэв. 9-р долоо хоногт `v1.0` tag ба GitHub Release нийтэлнэ.

5.8 Source

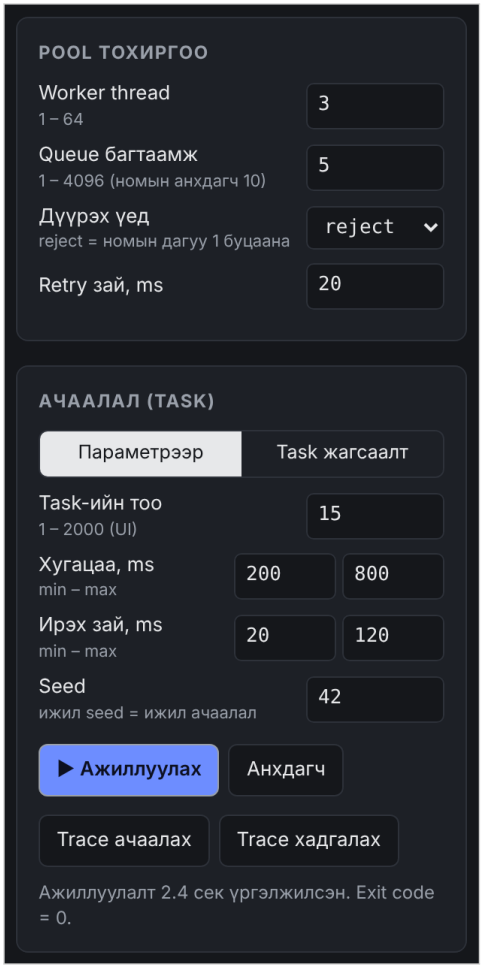
Номын API-ийн нэр, `client.c` жишээний бүтцийг сурах бичгээс авсан. Код, интерфейс, тест, баримт бичгийн анхны хувилбарыг AI туслахын тусламжтайгаар бичиж, баг бүх өөрчлөлтийг бодит C программ дээр ажиллуулан тест болон гараар шалгасан. Гадны сан, бэлэн component ашиглаагүй. Дэлгэрэнгүйг README-ийн ”AI ба бэлэн код ашигласан тухай” хэсгээс харна.

ЗУРГАА. ТУРШИЛТ БА UI ДҮРСЛЭЛ

6.1 Өөрчилж турших параметр

Хүснэгт 4. Тохиргооны параметр

Параметр	Нөлөө	Хүрээ (анхдагч)
Worker thread	зэрэг ажиллах thread-ийн тоо; timeline-ийн мөрийн тоо	1-64 (3)
Queue багтаамж	хүлээж болох task-ийн тоо; бага бол татгалзал олширно	1-4096 (5)
Дүүрэх үед	reject: номын дагуу 1 буцааж хаяна; retry: client хүлээгээд дахин оролдоно	reject
Retry зай (мс)	retry горимд дахин оролдох хоорондын зай	1-10000 (20)
Task-ийн тоо	үүсгэх task-ийн тоо	1-2000 (15)
Хугацаа (мс)	task бүрийн ажиллах хугацааны муж	1-60000 (200-800)
Ирэх зай (мс)	дараалсан task-уудын хоорондох зай; 0 бол бүгд зэрэг	0-60000 (20-120)
Seed	ижил seed = ижил ачаалал	≥ 0 (42)
Task жагсаалт	мөр бүр `arrival_ms duration_ms`; бэлэн тест сонгож болно	≤ 2000 мөр



Зураг 4. Тохиргоо: pool, ачаалал, ажиллуулах товч

6.2 Юуг ажиглах

- Queue хоосон үед worker-ууд ”○ idle — sem_wait” төлөвтэй, timeline дээр судалтай саарал байна; task ирмэгц нэг нь сэрж running болно.
- Worker бүх нь завгүй үед task-ууд queue-д хуримтлагдаж, queue-ийн уртын график өснө; task хамгийн зүүн (head) талаас нь авагдана (FIFO).
- Queue дүүрсэн үед client мөрөнд улаан ✕ гарч, ”татгалзсан” тоолуур өснө (`pool_submit` 1 буцаасан).
- Нэг worker нэг мөчид зөвхөн нэг task-тай; нэгэн зэрэг ажиллах task-ийн тоо worker-ийн тооноос хэтрэхгүй.
- Shutdown (тасархай зураас)-ы дараа queue-д үлдсэн task-ууд гүйцэтгэгдэж, дараа нь worker бүр гарна (бараан бүс).

6.3 Бэлэн туршилтууд

Хүснэгт 5. Интерфейсийн бэлэн туршилт

Туршилт	Тохиргоо	Хүлээгдсэн ажиглалт
`basic.txt`	W=3, Q=10, 10 task	бүх task хүлээн авагдаж, зэрэг ≤ 3
`burst.txt`	W=2, Q=3, 10 task t=0, reject	3-5 хүлээн авч, үлдсэнийг татгалзана
`burst.txt`	W=2, Q=3, retry	10 бүгд хүлээн авагдана, нийт хугацаа уртасна
`fifo.txt`	W=1, Q=10, 6 task	ирсэн дарааллаараа T0…T5
`mixed_long_short.txt`	W=3, 1×1500 мс + 7×100 мс	урт task нэг worker-ийг эзэлж, бусад нь богинуудыг гүйцэтгэнэ

ДОЛОО. ТУРШИЛТЫН АРГА

7.1 Орчин

Хөгжүүлэлт ба автомат туршилт: Ubuntu 24.04.5 LTS (x86_64, kernel 6.18.44-fc-v185), gcc 13.3.0, glibc 2.39, Python 3.13.18, CPU: Intel(R) Xeon(R) Processor @ 2.80GHz, 2 core. Интерфейсийг Chromium (headless) дээр туршиж дэлгэцийн зургийг авав. CI: GitHub Actions ubuntu-latest (gcc), macos-latest (clang). Windows дээр WSL2 Ubuntu-аар ажиллана.

Үүнээс гадна багийн гишүүн Ц.Номинжингийн Ubuntu компьютер дээр Manual-ын дагуу repository-г татаж, VS Code-ийн терминал болон дотоод browser-ооc бүх командыг дахин ажиллуулж шалгав (8.4 хэсэг).

7.2 Core-ийн автомат тест

`tests/run_tests.sh` нь программыг олон тохиргоогоор ажиллуулж, гарсан trace-ийг шалгана:

- **Номын жишээ (T1)**: `examples/client.c` 5 нийлбэрийг зөв хэвлэх.
- **Үндсэн (T2-T7)**: анхдагч тохиргоо, queue дүүрэх (reject, retry), FIFO, урт ба богино task, санамсаргүй ачаалал.
- **Захын (B1-B5)**: queue=1, worker > task, 64 worker, 1 task, 500 task-ийн stress.
- **Буруу оролт (E1-E17)**: 0 ба 65 worker, queue=0, тоо биш утга, MIN > MAX, сөрөг зай, үл мэдэгдэх бодлого/сонголт, утгагүй сонголт, байхгүй файл, файлд тоо биш утга, эрэмбэгүй мөр, 3 баганатай мөр, 0 хугацаа, хоосон файл. Exit код 2 ба алдааны текстийг шалгана.

7.3 Trace шалгагч ба ThreadSanitizer

`tests/verify_trace.py` нь C кодоос хараат бусаар: queue-ийн уртыг SUBMIT/START-аас дахин сэргээж бүртгэсэн q-тэй тулгах; нэг worker дээрх интервал давхцахгүй; START-ийн дараалал = SUBMIT-ийн дараалал (FIFO); SHUTDOWN-ы дараа SUBMIT байхгүй; worker бүр WORKER_EXIT хийсэн эсэхийг шалгана. Мөн тест бүрт хүлээгдсэн тоог (`--expect accepted=10` гэх мэт) тулгана. `make tsan` нь `-fsanitize=thread`-тэй build хийж, 4 worker, queue 3, 40 task, retry горимоор өрсөлдөөнийг ихэсгэн ажиллуулна.

7.4 Гүйцэтгэлийн туршилт

`tests/experiments.sh` нь worker-ийн тоо, queue-ийн багтаамж, дүүрэх үеийн бодлогыг тус тусад нь өөрчилж, бусад параметрийг тогтмол барина (seed тогтмол).

7.5 Гар туршилт

Интерфейсийн бэлэн туршилтууд, буруу оролт (worker 0, task жагсаалтад тоо биш утга), алхамчлах, хурд солих, trace хадгалах ба сервергүйгээр ачаалахыг browser дээр гараар туршиж, дэлгэцийн зургаар баримтжуулав (8-р бүлэг).

НАЙМ. ҮР ДҮН

8.1 Core-ийн тестийн үр дүн

Linux дээрх ажиллалтын үр дүн (Хүснэгт 6). Бодит үр дүн баганын тоо `build/test_output.txt`-ээс шууд авсан.

Хүснэгт 6. Автомат тестийн үр дүн (`make test`, `make tsan`)

ID	Тест	Бодит үр дүн	Дүн
T1	Номын `client.c`	15, 3, 300, 0, 42 зөв хэвлэгдсэн	PASS
T2	Номын анхдагч 3 worker, queue 10, 10 task	хүлээн авсан 10, татгалзсан 0, зэрэг 3, queue 4	PASS
T3	Burst: 10 task t=0, 2 worker, queue 3, reject	хүлээн авсан 3, татгалзсан 7, зэрэг 2, queue 3	PASS
T4	Burst, retry горим	хүлээн авсан 10, татгалзсан 0, зэрэг 2, queue 3	PASS
T5	1 worker: FIFO дараалал	хүлээн авсан 6, татгалзсан 0, зэрэг 1, queue 5	PASS
T6	1 урт + 7 богино task, 3 worker	хүлээн авсан 8, татгалзсан 0, зэрэг 3, queue 5	PASS

ID	Тест	Бодит үр дүн	Дүн
T7	4 worker, queue 8, 20 санамсаргүй task	хүлээн авсан 20, татгалзсан 0, зэрэг 4, queue 5	PASS
B1	queue=1, 1 worker, 6 task зэрэг	хүлээн авсан 1, татгалзсан 5, зэрэг 1, queue 1	PASS
B2	8 worker, 3 task (worker > task)	хүлээн авсан 3, татгалзсан 0, зэрэг 3, queue 3	PASS
B3	64 worker (дээд хязгаар), 64 task	хүлээн авсан 64, татгалзсан 0, зэрэг 64, queue 27	PASS
B4	1 task, 1 мс	хүлээн авсан 1, татгалзсан 0, зэрэг 1, queue 1	PASS
B5	500 task, 8 worker, queue 16, retry	хүлээн авсан 500, татгалзсан 0, зэрэг 8, queue 16	PASS
E1-E17	Буруу оролт	exit=2, тохирох тайлбар (жишээ нь ”-w буруу (’0’): worker-ийн тоо 1..64 бүхэл тоо байна”)	PASS
R1	ThreadSanitizer	race илрээгүй, VERIFY: OK	PASS

Нийт 29/29 PASS. Бүх тестийн trace хараат бус шалгагчаар баталгаажсан, Thread-Sanitizer race илрүүлээгүй. B3 тестэд 64 worker бүгд зэрэг ажилласан, B5 тестэд 500 task бүр яг 1 удаа гүйцэтгэгдсэн.

8.2 Гүйцэтгэлийн туршилтын үр дүн

Туршилт 1. Worker-ийн тоо. 40 task, хугацаа 100-300 мс, бүгд t=0-д ирнэ, queue 64, seed 1.

Хүснэгт 7. Worker-ийн тооны нөлөө

Worker	Нийт хугацаа (мс)	Дундаж хүлээлт (мс)	Дундаж turnaround (мс)	Ашиглалт (%)	Хурдатгал
1	8377	4021	4230	100.0	1.00×
2	4223	1955	2164	99.2	1.98×
4	2196	921	1131	95.3	3.81×
8	1139	406	616	91.9	7.35×
16	647	151	360	81.0	12.96×

Нийт хугацаа worker-ийн тоонд бараг урвуу пропорциональ буурсан (2 worker → 1.98×, 8 worker → 7.35×). 16 worker-т ашиглалт 81% хүртэл буурсан: 40 task 16 worker-т тэгш хуваагдахгүй тул сүүлийн давалгаанд зарим worker idle үлдэнэ.

Туршилт 2. Queue-ийн багтаамж. 2 worker, 20 task t=0-д, тус бүр 100 мс, reject.

Хүснэгт 8. Queue-ийн багтаамжийн нөлөө

Queue	Хүлээн авсан	Татгалзсан	Queue дээд урт	Нийт хугацаа (мс)
1	1	19	1	100
2	2	18	2	100
4	4	16	4	201
8	8	12	8	401
16	16	4	16	802
20	20	0	20	1002

Хүлээн авсан task-ийн тоо багтаамжтай яг тэнцүү гарсан. Client 20 task-ийг микросекундын дотор submit хийдэг бол worker `sem_wait`-аас сэрж dequeue хийхэд түүнээс урт хугацаа шаардана; иймээс burst-ийн үед queue хоосрохоос өмнө дүүрнэ.

Туршилт 3. Дүүрэх үеийн бодлого. 2 worker, queue 3, 20 task 10 мс зайтай, тус бүр 100 мс.

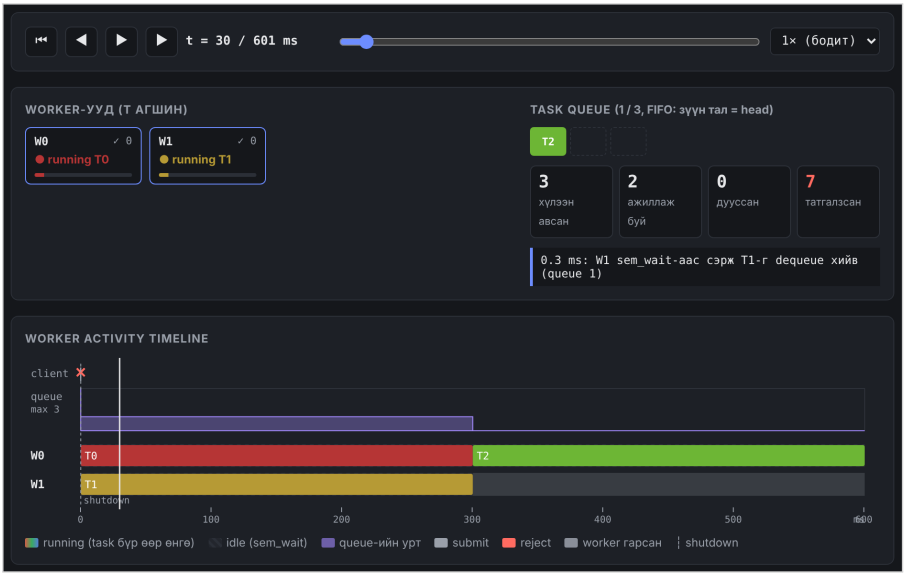
Хүснэгт 9. Reject ба retry бодлого

Бодлого	Хүлээн авсан task	Татгалзсан оролдлого	Нийт хугацаа (мс)	Дундаж хүлээлт (мс)
reject	7	13	401	89
retry	20	38	1012	120

Reject горимд task алдагдсан ч нийт хугацаа богино; retry горимд бүх task гүйцэтгэгдсэн боловч client хүлээсэн тул уртассан.

8.3 Интерфейсийн үр дүн

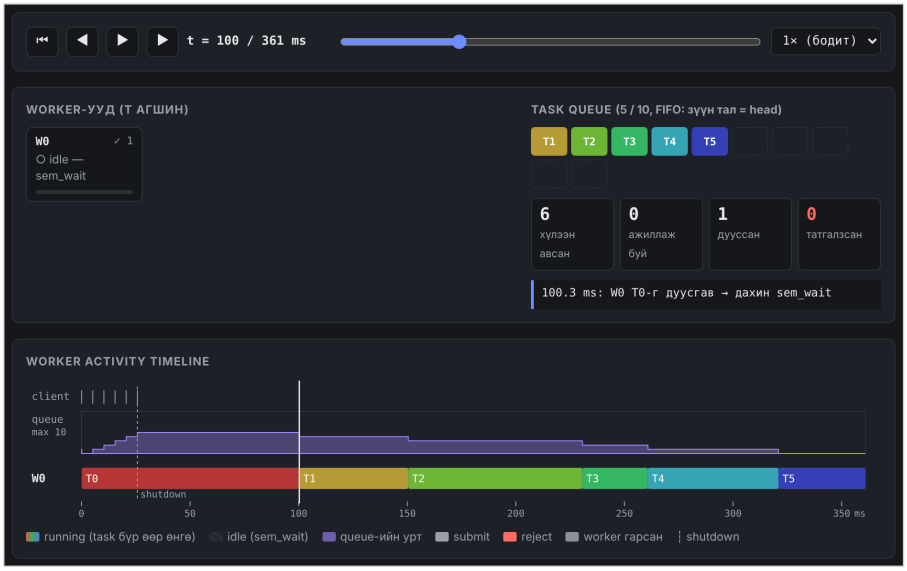
Зураг 5-12 нь бодит программын ажиллалтаас авсан дэлгэцийн зураг.



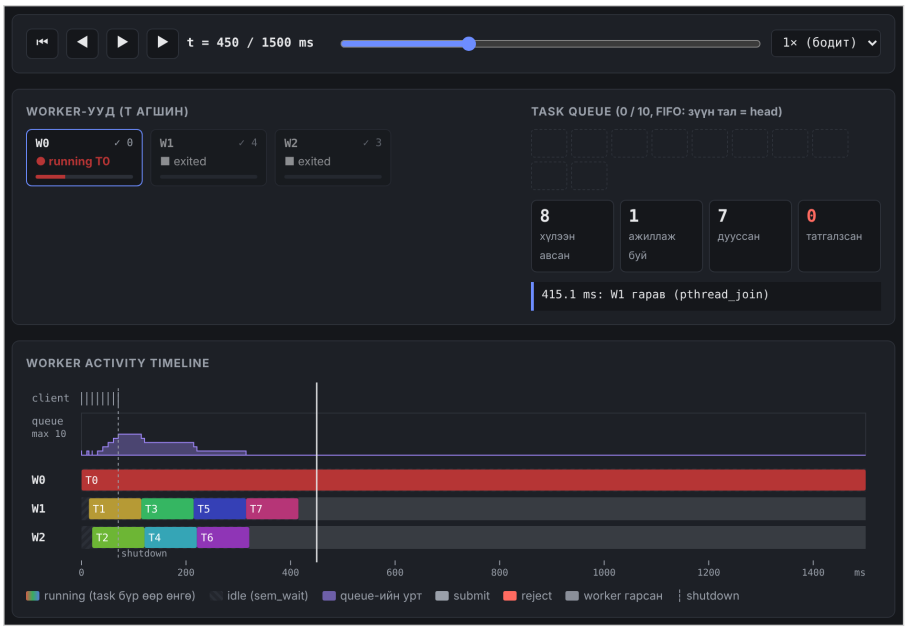
Зураг 5. Queue дүүрэх (W=2, Q=3, reject): t=30 мс-д T0, T1 ажиллаж, T2 queue-д, 7 task татгалзсан (client мөрөнд ✕)



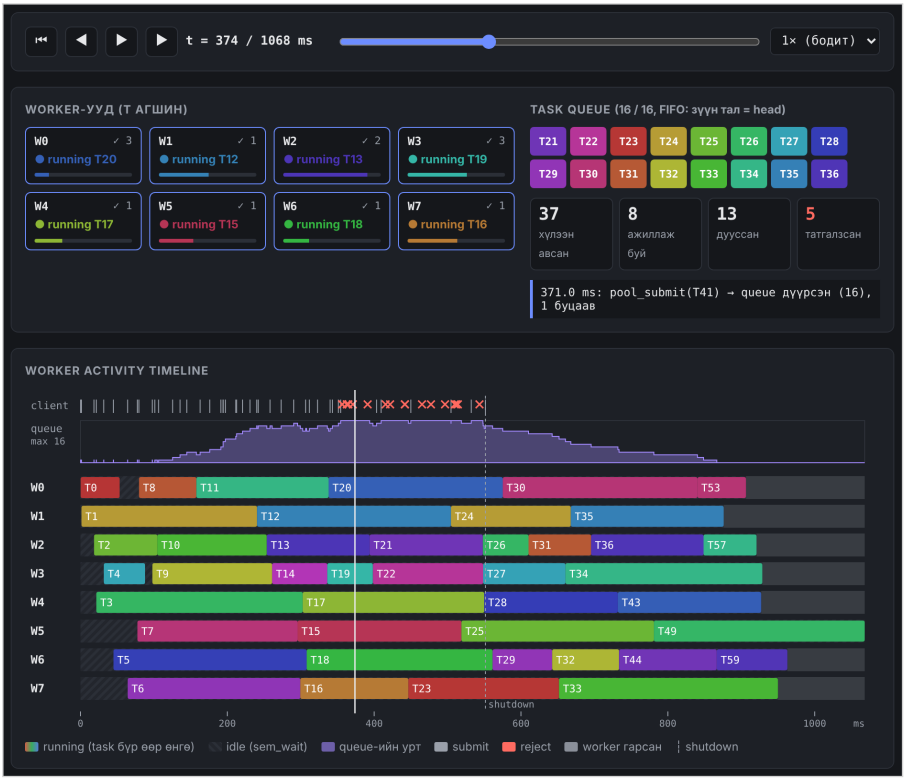
Зураг 6. Ижил ачаалал, retry горим: client дахин оролдож (✕ олон), 10 task бүгд гүйцэтгэгдсэн



Зураг 7. FIFO (W=1): дараагийн үйлдэл товчоор алхамчилсан; T3, T4, T5 ирсэн дарааллаараа queue-д хүлээж байна



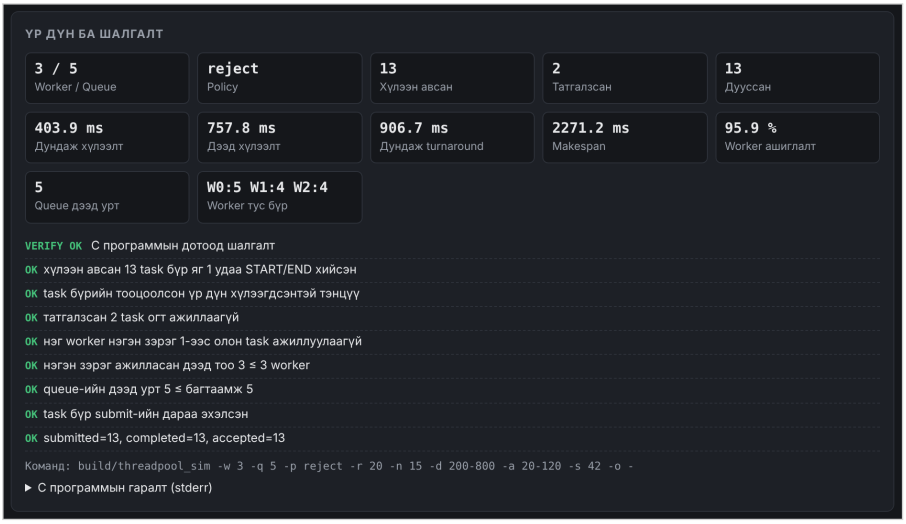
Зураг 8. Урт (T0 = 1500 мс) ба богино task: W0 урт task-аар эзлэгдсэн ч W1, W2 богинуудыг гүйцэтгэсээр



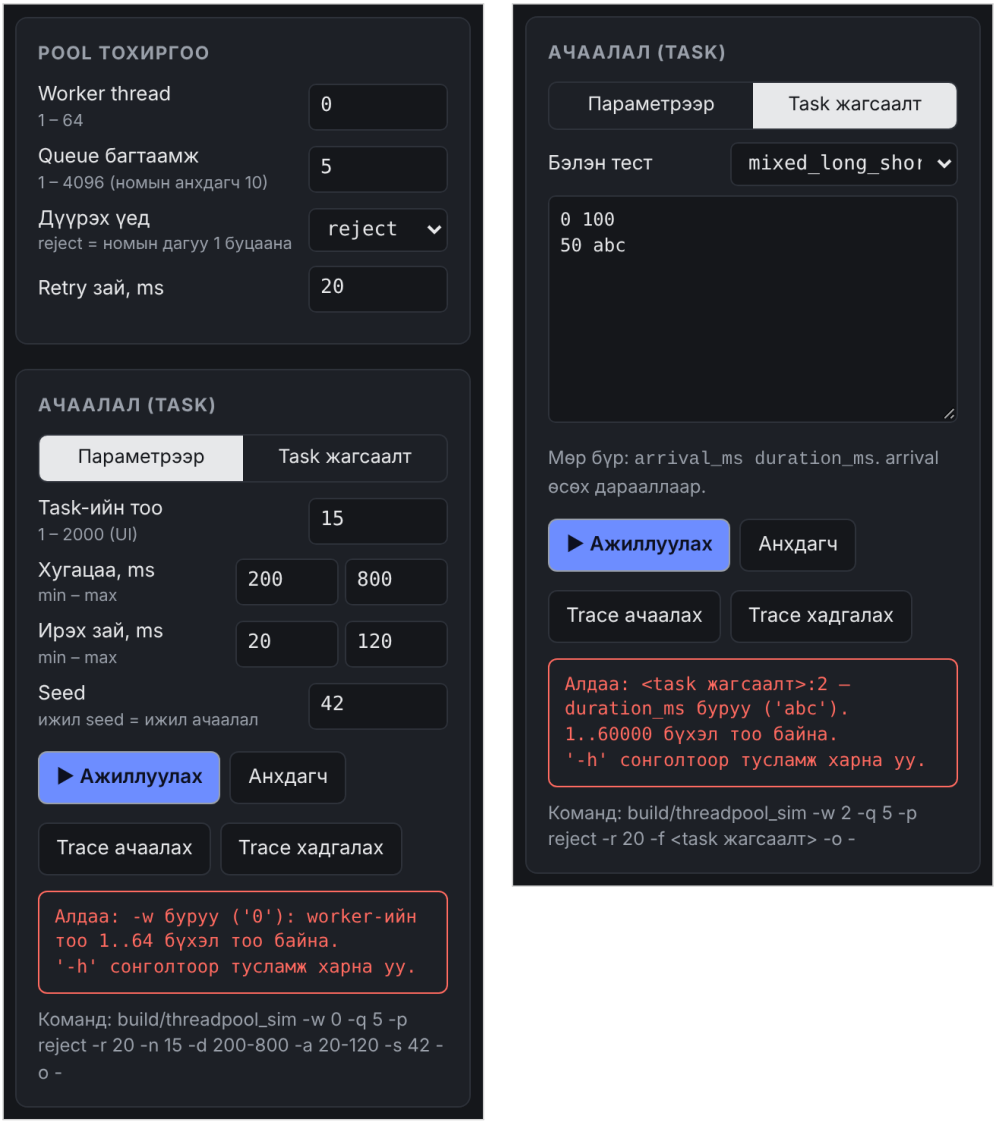
Зураг 9. 8 worker, queue 16, 60 task: queue-ийн урт ачааллын давалгааг дагаж өсөж буурна



Зураг 10. Ижил 12 task (100-300 мс, seed 1): дээр 1 worker, доор 4 worker; нийт хугацаа ойролцоогоор 4 дахин богиносов



Зураг 11. Үр дүн ба шалгалт: хэмжүүрүүд, VERIFY OK ба 8 шалгалт, C программын команд



Зураг 12. Буруу оролт: зүүн талд worker = 0, баруун талд task жагсаалтын 2-р мөрөнд тоо биш утга

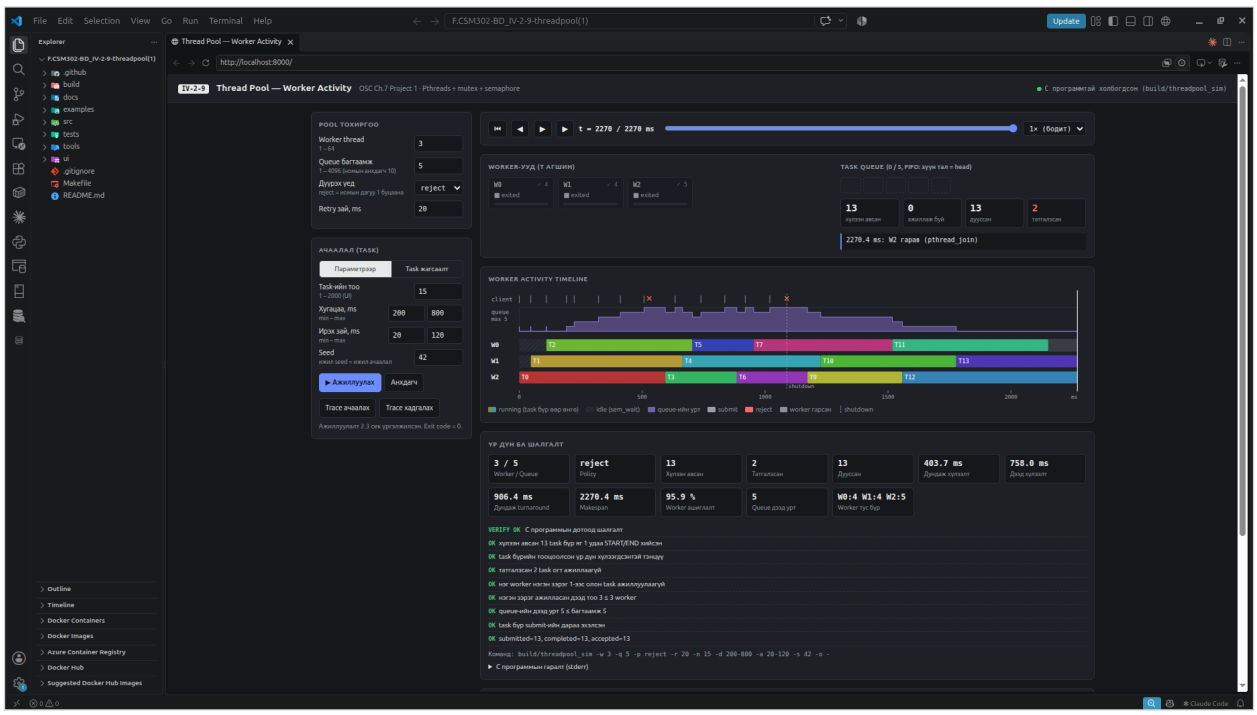
8.4 Хэрэглэгчийн компьютер дээрх баталгаажуулалт

Программ зөвхөн хөгжүүлэлтийн орчинд биш, өөр компьютер дээр ч Manual-ын дагуу ажилладаг эсэхийг шалгахын тулд багийн гишүүний Ubuntu компьютер дээр (`nominjin@ubuntu`) repository-г zip-ээр татаж, VS Code-оос `make`, `make test`, `make ui`, `make tsan` болон CLI командуудыг ажиллуулав. Нэмэлт тохиргоо, засвар хийгээгүй. Үр дүнг Хүснэгт 10-д нэгтгэв.

Хүснэгт 10. Хэрэглэгчийн компьютер дээрх ажиллалтын үр дүн

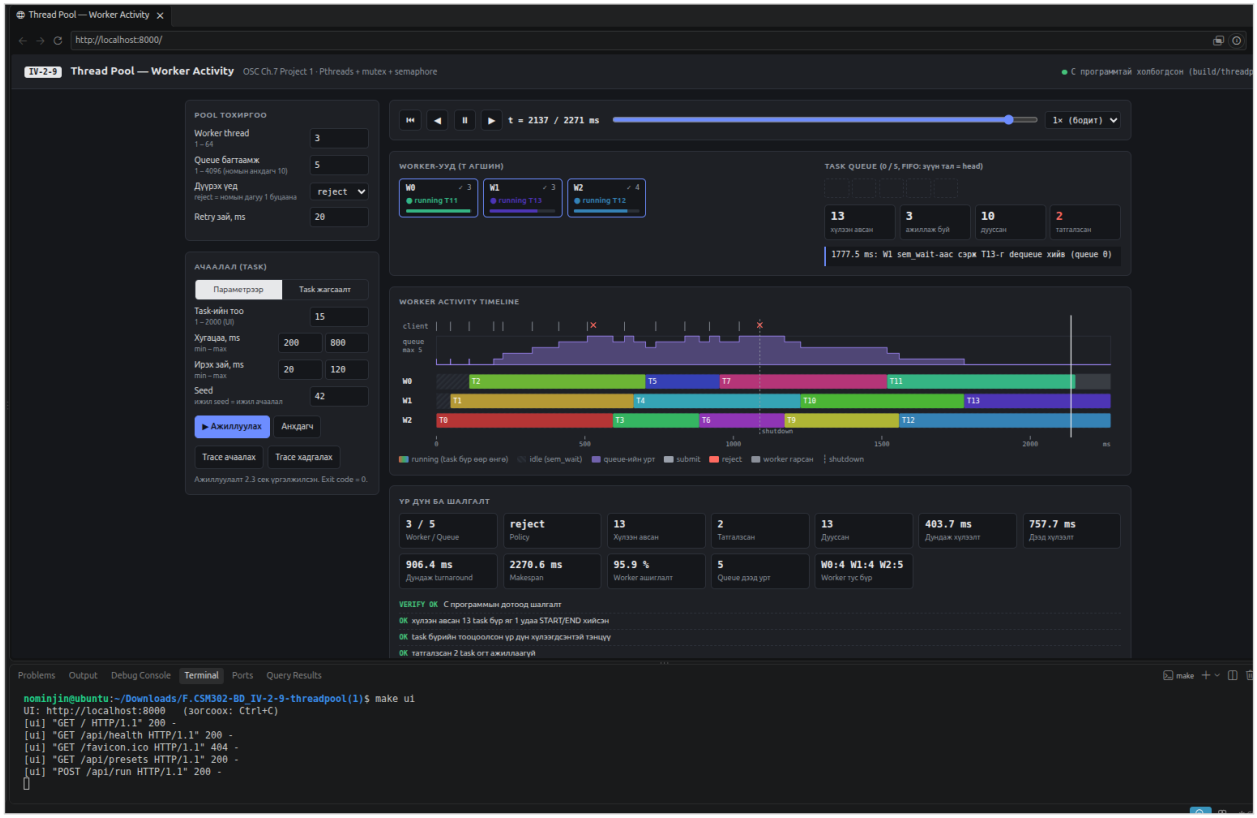
Шалгалт	Хүлээгдсэн үр дүн	Хэрэглэгчийн компьютер дээр	Зураг
`make test`	29 PASS, 0 FAIL	29 PASS, 0 FAIL	15
`make ui`	С программтай холбогдож, run хийгдэнэ	”С программтай холбогдсон”, POST /api/run 200, VERIFY OK	13, 14
`./build/client`	5 нийлбэр зөв	3, 0, 42, 15, 300 (дараалал өөр)	16
`-w 3 -q 5 -n 15`	15/15 гүйцэтгэгдэнэ	15 хүлээн авсан, 0 татгалзсан, 8 шалгалт OK	16
`-w 2 -q 3 -f burst.txt`	3-5 хүлээн авч, үлдсэнийг татгалзана	3 хүлээн авсан, 7 татгалзсан	16
`make tsan`	race илрэхгүй	ThreadSanitizer WARNING байхгүй, VERIFY OK	17

Зураг 13-д VS Code-ийн дотоод browser-оос `http://localhost:8000` нээж, анхдагч тохиргоогоор (3 worker, queue 5, 15 task, seed 42) ажиллуулаад төгсгөлийн агшинд (`t = 2270 ms`) очсон байдал харагдана. Shutdown-ы дараа гурван worker бүгд ”exited” төлөвтэй, queue хоосон, 13 task хүлээн авч, 2 task татгалзсан, бүх шалгалт OK. Seed ижил тул ачаалал 8.3 хэсгийн ажиллалттай (Зураг 11) ижил: хүлээн авсан, татгалзсан тоо (13, 2) яг давхцаж, нийт хугацаа (2270.4 мс ба 2271.2 мс) 1 мс-ийн дотор зөрсөн. Харин worker бүрийн дуусгасан task-ийн тоо (W0:4, W1:4, W2:5 ба W0:5, W1:4, W2:4) өөр байгаа нь аль worker аль task-ийг авахыг OS scheduler шийддэгийг харуулна. Тиймээс үр дүн өөр компьютер дээр давтагдаж, зөвхөн хуваарилалт л өөрчлөгдөнө.



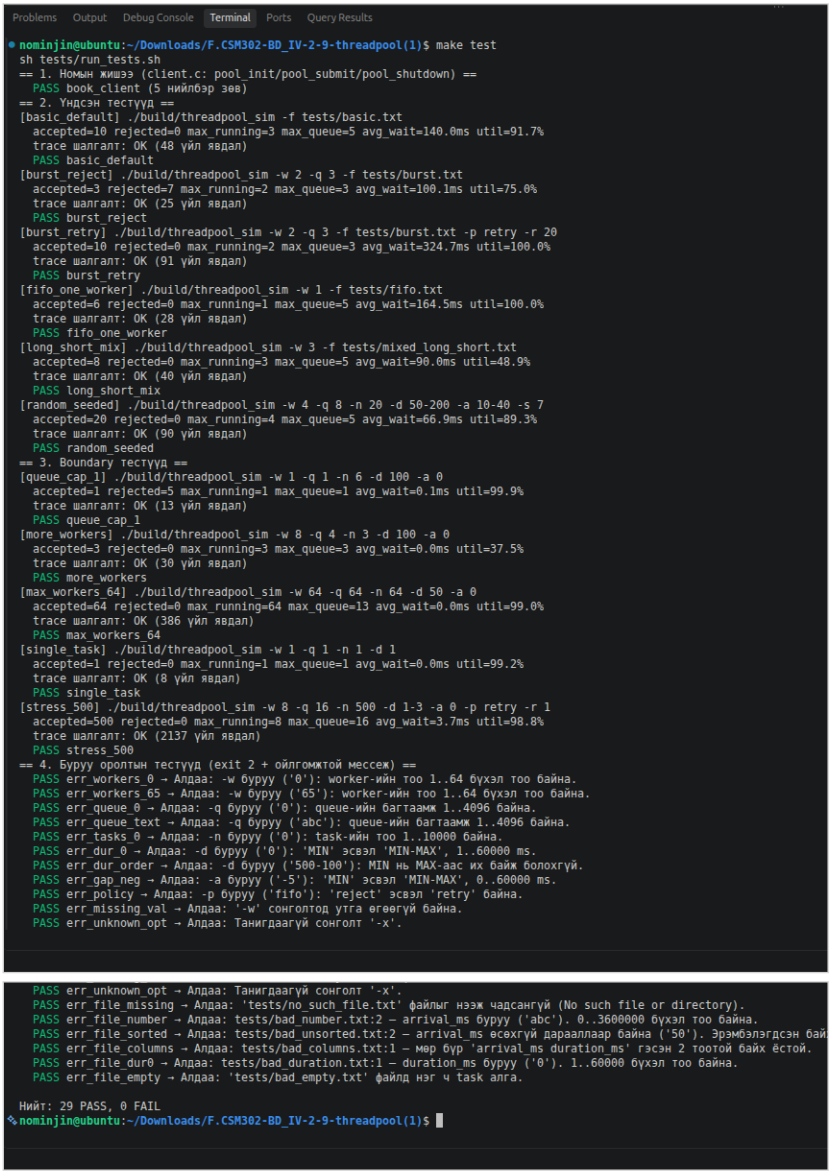
Зураг 13. Хэрэглэгчийн компьютер: VS Code-ийн browser дээр Worker activity, ажиллалтын төгсгөл (бүх worker exited, VERIFY OK)

Зураг 14-д playback-ийг `t = 2137 ms` агшинд зогсоосон: W0, W1, W2 нь T11, T13, T12-ийг гүйцэтгэж (running), queue хоосон, 10 task дууссан. Доорх терминалд `make ui`-ийн серверийн бүртгэл харагдана: браузер `GET /`, `/api/health`, `/api/presets` хүсэлтийн дараа `POST /api/run` илгээж, сервер С программыг ажиллуулаад 200 кодоор trace буцаасан. Өөрөөр хэлбэл дэлгэцийн дүрслэл бодит С программын ажиллалтаас гарсан. `/favicon.ico`-ийн 404 нь хуудасны дүрс тэмдэг байхгүйг заах бөгөөд ажиллагаанд нөлөөлөхгүй.



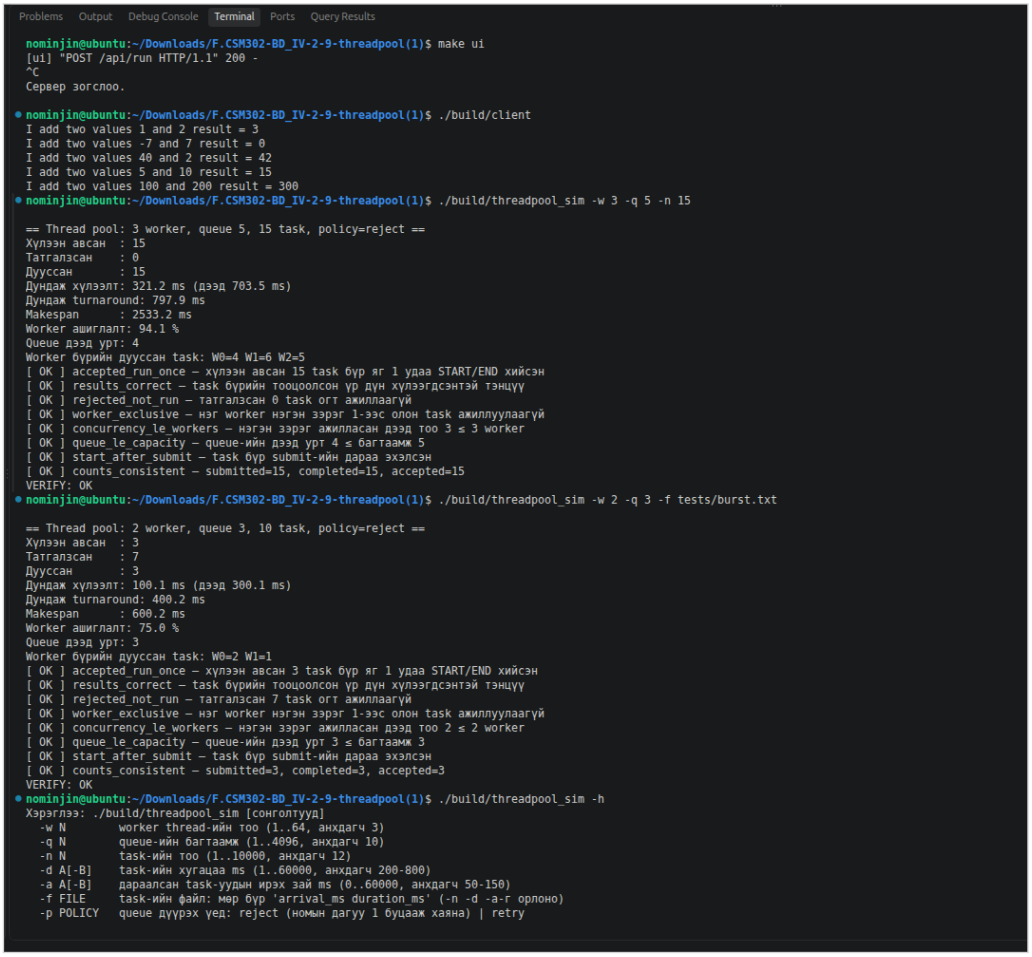
Зураг 14. Ажиллалтын дунд агшин (3 worker running) ба серверийн бүртгэл (POST /api/run 200)

Зураг 15-д `make test`-ийн бүрэн гаралт: номын жишээ, 6 үндсэн, 5 захын тест бүр С талын шалгалт болон хараат бус trace шалгагчаар (”trace шалгалт: OK”) баталгаажиж, 17 буруу оролтын тест бүр зөв алдааны мессеж өгсөн. Төгсгөлд `Нийт: 29 PASS, 0 FAIL`. Тоон утгууд (хүлээлт, ашиглалт) Хүснэгт 6-ийнхтэй ойролцоо, хүлээн авсан/татгалзсан тоо ижил.



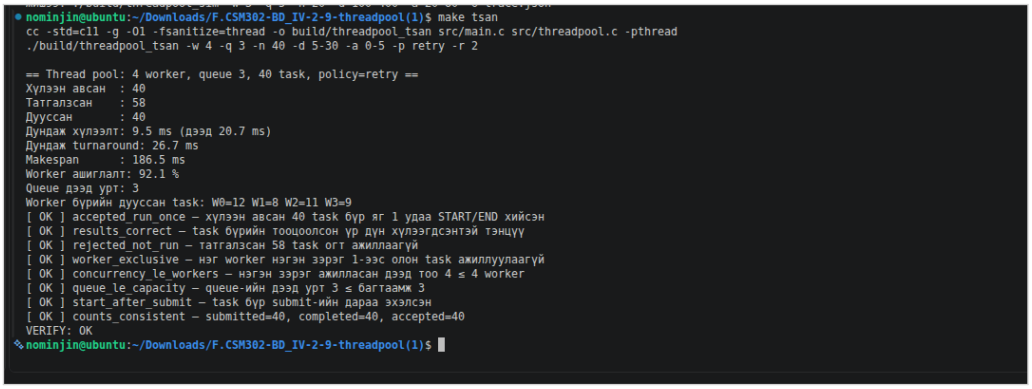
Зураг 15. Хэрэглэгчийн компьютер дээрх make test: 29 PASS, 0 FAIL

Зураг 16-д серверийг `Ctrl+C`-ээр зогсоосны (”Сервер зогслоо”) дараа CLI командуудыг шууд ажиллуулсан. `./build/client`-ийн 5 нийлбэр зөв боловч хэвлэгдсэн дараалал (3, 0, 42, 15, 300) өмнөх ажиллалтаас (15, 3, 300, 0, 42) өөр байна. Энэ нь гурван worker зэрэгцэн ажиллаж, аль нь түрүүлж дуусахыг OS scheduler шийддэгийн бодит жишээ бөгөөд алдаа биш. `-w 3 -q 5 -n 15` ажиллалтад 15 task бүгд гүйцэтгэгдэж, burst тестэд queue (3) дүүрч 7 task татгалзагдсан нь номын ”дүүрвэл 1 буцаана” дүрмийг харуулна. Төгсгөлд `-h` нь бүх сонголтын тайлбарыг хэвлэв.



Зураг 16. Серверийг зогсоох, номын client.c, анхдагч ба burst ажиллалт, -h тусламж

Зураг 17-д `make tsan`: программыг `-fsanitize=thread`-тэй хөрвүүлж, 4 worker, queue 3, 40 task-ийг retry горимоор маш богино (5-30 мс) хугацаатай ажиллуулсан. Queue байнга дүүрч client 58 удаа дахин оролдсон ч ThreadSanitizer нэг ч `WARNING` гаргаагүй, 40 task бүр яг нэг удаа гүйцэтгэгдэж `VERIFY: OK`. Энэ нь өрсөлдөөн ихтэй нөхцөлд ч mutex болон semaphore queue-г зөв хамгаалж байгааг баталгаажуулна.



Зураг 17. Хэрэглэгчийн компьютер дээрх ThreadSanitizer: race илрээгүй, VERIFY OK

ЕС. ХЭЛЭЛЦҮҮЛЭГ

9.1 Үр дүнгийн тайлбар

Бүх тестэд, тэр дундаа 64 worker зэрэг ажилласан болон 500 task-ийн stress нөхцөлд task бүр яг нэг удаа ажиллаж, нэг worker давхар task аваагүй, queue FIFO-оор ажиллаж, багтаамжаас хэтрээгүй нь mutex queue-ийн өгөгдлийг, semaphore task-ийн тоог зөв хамгаалж байгааг харуулна. Idle worker-ууд `sem_wait` дээр унтдаг тул CPU үрэхгүй; timeline-ийн судалтай саарал бүс тэдгээрийн бодит хүлээлт юм. Task-ууд `nanosleep` хийдэг (I/O-той адил) учраас worker-ийн тоо CPU-ийн цөмийн тооноос олон байсан ч хурдатгал бараг шугаман хэвээр байсан.

Ижил параметрээр дахин ажиллуулахад аль worker аль task-ийг авах, burst үед хэдэн task хүлээн авагдах нь бага зэрэг өөрчлөгдөж болно (T3: 3-5). Энэ нь OS scheduler-ийн шийдвэр бөгөөд тест ийм хэлбэлзлийг мужаар шалгадаг.

9.2 Хүлээх дараалал ба deadlock

Worker эхлээд `sem_wait`, дараа нь `pthread_mutex_lock` хийх ёстой. Хэрэв дарааллыг солиод mutex-ийг барьсан чигээр хоосон queue дээр `sem_wait` хийвэл client mutex авч чадахгүй, task нэмэгдэхгүй тул хэн ч урагшлахгүй (deadlock). Мөн client `sem_post`-ийг mutex суллагдсаны дараа хийдэг. Mutex-ийг зөвхөн queue-г өөрчлөх богино хэсэгт барьж, task-ийг гүйцэтгэх болон хүлээлтийг critical section-ээс гадуур хийсэн.

9.3 Хязгаарлалт

- Task-ийн ажлыг `nanosleep`-ээр дуурайлгасан (номын зөвлөмж). CPU-д ачаалалтай ажилд хурдатгал цөмийн тоогоор хязгаарлагдана.
- Trace бүртгэл тусдаа mutex ашигладаг тул маш богино (< 1 мс) task-уудад бага зэргийн нэмэлт зардал үүсгэнэ.
- Нэг client thread submit хийдэг; retry горимд түүний хүлээлт дараагийн task-уудын ирэлтийг хойшлуулна.
- Номын API глобал төлөвтэй тул нэг процесст нэг pool ажиллана.
- Интерфейсийн хязгаар (2000 task, ~60 с) C программынхаас (10000 task) нарийн.

АРАВ. ДҮГНЭЛТ

Сурах бичгийн Programming Project 1-ийг C хэл, pthreads, POSIX semaphore, mutex ашиглан бүрэн хэрэгжүүлж, worker-ийн тоо, queue-ийн багтаамж, дүүрэх үеийн бодлого, ачааллыг өөрчилж турших боломжтой болгосон. Номын `client.c` жишээ өөрчлөлтгүй ажиллав. Багшийн оноосон ”Worker activity” дүрслэлийг бодит программын trace-тэй шууд холбож, worker бүрийн idle, running, completed төлөв, queue, татгалзлыг алхам алхмаар харуулав. 29 автомат тест, хараат бус trace

шалгагч, ThreadSanitizer зөв ажиллагааг нотолж, гүйцэтгэлийн туршилт worker нэмэхэд хурдатгал бараг шугаман байгааг харуулсан. Manual-ын дагуу `make`, `make ui` гэсэн хоёр командаар өөр компьютерт ажиллуулах боломжтой.

АШИГЛАСАН МАТЕРИАЛ

[1] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Wiley, 2018. 4.5.1 (Thread pools), 4.6.3 (Thread cancellation), 6.5–6.6, 7.1.1, 7.3; Chapter 7 Programming Project 1 ”Designing a Thread Pool”.

[2] The Open Group. POSIX.1-2017: `pthread_create`, `pthread_join`, `pthread_mutex_lock`, `sem_init`, `sem_wait`, `sem_post`.

[3] Linux man-pages: `sem_overview(7)`, `pthreads(7)`, `nanosleep(2)`.

[4] ThreadSanitizer. `https://clang.llvm.org/docs/ThreadSanitizer.html`

[5] F.CSM302 Бие даалт 1 удирдамж, Сэдэв ба шүүмжийн хуваарилалт.

[6] Багийн repository. `github.com/NominjinNomio/F.CSM302-BD_IV-2-9-threadpool`

ХАВСРАЛТ А. АЖЛЫН ТӨЛӨВЛӨГӨӨ

Талбар	Мэдээлэл
Хуваарилалтын код	IV-2-9
Төсөл	Designing a Thread Pool
Repository	`https://github.com/NominjinNomio/F.CSM302-BD_IV-2-9-threadpool`
Хэл/сан/орчин	C (pthreads, POSIX semaphore), Python 3, HTML/JavaScript; Linux, macOS, WSL2
Үндсэн дүрслэл	Worker activity

Гишүүн	Хариуцах ажил	Commit/file
Ц.Номинжин /B241940137/	Сервер, CI, screenshot	`tools/, .github/`
Ц.Номинжин /B241940137/	Туршилтын программ, CLI, оролтын шалгалт	`src/main.c, examples/`
Ц.Номинжин /B241940137/	Автомат тест, trace шалгагч, туршилт	`tests/`
Ц.Номинжин /B241940137/	Thread pool core: queue, mutex, semaphore, worker	`src/threadpool.c, src/threadpool.h`
Ц.Номинжин /B241940137/	Worker activity интерфейс	`ui/index.html`
Ц.Номинжин /B241940137/	Тайлан, гарын авлага	`docs/, README.md`

Тест	Оролт/нөхцөл	Хүлээгдсэн үр дүн
1	`./build/client` (номын жишээ)	5 нийлбэр зөв
2	W=3, Q=10, basic.txt	10/10 хүлээн авна, зэрэг ≤ 3
3	W=2, Q=3, burst.txt (queue дүүрэх)	3-5 хүлээн авна, үлдсэнд `pool_submit` 1 буцаана
4	W=1, fifo.txt	FIFO дараалал
5	W=0, буруу файл	exit 2, алдаатай утга/мөрийг заасан тайлбар